

要理解 `r` 的类型到底是什么，最简单的办法是从右向左阅读 `r` 的定义。离变量名最近的符号（此例中是 `&r` 的符号 `&`）对变量的类型有最直接的影响，因此 `r` 是一个引用。声明符的其余部分用以确定 `r` 引用的类型是什么，此例中的符号 `*` 说明 `r` 引用的是一个指针。最后，声明的基本数据类型部分指出 `r` 引用的是一个 `int` 指针。



面对一条比较复杂的指针或引用的声明语句时，从右向左阅读有助于弄清楚它的真实含义。

2.3.3 节练习

59

练习 2.25: 说明下列变量的类型和值。

(a) `int* ip, i, &r = i;` (b) `int i, *ip = 0;` (c) `int* ip, ip2;`

2.4 const 限定符



有时我们希望定义这样一种变量，它的值不能被改变。例如，用一个变量来表示缓冲区的大小。使用变量的好处是当我们觉得缓冲区大小不再合适时，很容易对其进行调整。另一方面，也应随时警惕防止程序一不小心改变了这个值。为了满足这一要求，可以用关键字 **const** 对变量的类型加以限定：

```
const int bufSize = 512;    // 输入缓冲区大小
```

这样就把 `bufSize` 定义成了一个常量。任何试图为 `bufSize` 赋值的行为都将引发错误：

```
bufSize = 512;             // 错误：试图向 const 对象写值
```

因为 `const` 对象一旦创建后其值就不能再改变，所以 `const` 对象必须初始化。一如既往，初始值可以是任意复杂的表达式：

```
const int i = get_size();   // 正确：运行时初始化
const int j = 42;          // 正确：编译时初始化
const int k;               // 错误：k 是一个未经初始化的常量
```

初始化和 const

正如之前反复提到的，对象的类型决定了其上的操作。与非 `const` 类型所能参与的操作相比，`const` 类型的对象能完成其中大部分，但也不是所有的操作都适合。主要的限制就是只能在 `const` 类型的对象上执行不改变其内容的操作。例如，`const int` 和普通的 `int` 一样都能参与算术运算，也都能转换成一个布尔值，等等。

在不改变 `const` 对象的操作中还有一种是初始化，如果利用一个对象去初始化另外一个对象，则它们是不是 `const` 都无关紧要：

```
int i = 42;
const int ci = i;          // 正确：i 的值被拷贝给了 ci
int j = ci;               // 正确：ci 的值被拷贝给了 j
```

尽管 `ci` 是整型常量，但无论如何 `ci` 中的值还是一个整型数。`ci` 的常量特征仅仅在执行改变 `ci` 的操作时才会发挥作用。当用 `ci` 去初始化 `j` 时，根本无须在意 `ci` 是不是一个常量。拷贝一个对象的值并不会改变它，一旦拷贝完成，新的对象就和原来的对象没什么关系了。

60 默认状态下, const 对象仅在文件内有效

当以编译时初始化的方式定义一个 const 对象时, 就如对 bufSize 的定义一样:

```
const int bufSize = 512; // 输入缓冲区大小
```

编译器将在编译过程中把用到该变量的地方都替换成对应的值。也就是说, 编译器会找到代码中所有用到 bufSize 的地方, 然后用 512 替换。

为了执行上述替换, 编译器必须知道变量的初始值。如果程序包含多个文件, 则每个用了 const 对象的文件都必须得能访问到它的初始值才行。要做到这一点, 就必须在每一个用到变量的文件中都有对它的定义 (参见 2.2.2 节, 第 41 页)。为了支持这一用法, 同时避免对同一变量的重复定义, 默认情况下, const 对象被设定为仅在文件内有效。当多个文件中出现了同名的 const 变量时, 其实等同于在不同文件中分别定义了独立的变量。

某些时候有这样一种 const 变量, 它的初始值不是一个常量表达式, 但又确实有必要在文件间共享。这种情况下, 我们不希望编译器为每个文件分别生成独立的变量。相反, 我们想让这类 const 对象像其他 (非常量) 对象一样工作, 也就是说, 只在一个文件中定义 const, 而在其他多个文件中声明并使用它。

解决的办法是, 对于 const 变量不管是声明还是定义都添加 extern 关键字, 这样只需定义一次就可以了:

```
// file_1.cc 定义并初始化了一个常量, 该常量能被其他文件访问
extern const int bufSize = fcn();
// file_1.h 头文件
extern const int bufSize; // 与 file_1.cc 中定义的 bufSize 是同一个
```

如上述程序所示, file_1.cc 定义并初始化了 bufSize。因为这条语句包含了初始值, 所以它 (显然) 是一次定义。然而, 因为 bufSize 是一个常量, 必须用 extern 加以限定使其被其他文件使用。

file_1.h 头文件中的声明也由 extern 做了限定, 其作用是指明 bufSize 并非本文件所独有, 它的定义将在别处出现。



如果想在多个文件之间共享 const 对象, 必须在变量的定义之前添加 extern 关键字。

2.4 节练习

练习 2.26: 下面哪些句子是合法的? 如果有不合法的句子, 请说明为什么?

(a) const int buf;

(b) int cnt = 0;

(c) const int sz = cnt;

(d) ++cnt; ++sz;



2.4.1 const 的引用

可以把引用绑定到 const 对象上, 就像绑定到其他对象上一样, 我们称之为对常量的引用 (reference to const)。与普通引用不同的是, 对常量的引用不能被用作修改它所绑定的对象:

```
const int ci = 1024;
const int &rl = ci; // 正确: 引用及其对应的对象都是常量
```

```

r1 = 42;                // 错误: r1 是对常量的引用
int &r2 = ci;            // 错误: 试图让一个非常量引用指向一个常量对象

```

因为不允许直接为 `ci` 赋值，当然也就不能通过引用去改变 `ci`。因此，对 `r2` 的初始化是错误的。假设该初始化合法，则可以通过 `r2` 来改变它引用对象的值，这显然是不正确的。

术语：常量引用是对 `const` 的引用

C++ 程序员们经常把词组“对 `const` 的引用”简称为“常量引用”，这一简称还是挺靠谱的，不过前提是你得时刻记得这就是个简称而已。

严格来说，并不存在常量引用。因为引用不是一个对象，所以我们没法让引用本身恒定不变。事实上，由于 C++ 语言并不允许随意改变引用所绑定的对象，所以从这层意义上理解所有的引用又都算是常量。引用的对象是常量还是非常量可以决定其所能参与的操作，却无论如何都不会影响到引用和对象的绑定关系本身。

初始化和对 `const` 的引用

2.3.1 节（第 46 页）提到，引用的类型必须与其所引用对象的类型一致，但是有两个例外。第一种例外情况就是在初始化常量引用时允许用任意表达式作为初始值，只要该表达式的结果能转换成（参见 2.1.2 节，第 32 页）引用的类型即可。尤其，允许为一个常量引用绑定非常量的对象、字面值，甚至是个一般表达式：

```

int i = 42;
const int &r1 = i;        // 允许将 const int& 绑定到一个普通 int 对象上
const int &r2 = 42;        // 正确: r1 是一个常量引用
const int &r3 = r1 * 2;    // 正确: r3 是一个常量引用
int &r4 = r1 * 2;         // 错误: r4 是一个普通的非常量引用

```

要想理解这种例外情况的原因，最简单的办法是弄清楚当一个常量引用被绑定到另外一种类型上时到底发生了什么：

```

double dval = 3.14;
const int &ri = dval;

```

此处 `ri` 引用了一个 `int` 型的数。对 `ri` 的操作应该是整数运算，但 `dval` 却是一个双精度浮点数而非整数。因此为了确保让 `ri` 绑定一个整数，编译器把上述代码变成了如下形式：

```

const int temp = dval;    // 由双精度浮点数生成一个临时的整型常量
const int &ri = temp;     // 让 ri 绑定这个临时量

```

62

在这种情况下，`ri` 绑定了一个临时量（temporary）对象。所谓临时量对象就是当编译器需要一个空间来暂存表达式的求值结果时临时创建的一个未命名的对象。C++ 程序员们常常把临时量对象简称为临时量。

接下来探讨当 `ri` 不是常量时，如果执行了类似于上面的初始化过程将带来什么样的后果。如果 `ri` 不是常量，就允许对 `ri` 赋值，这样就会改变 `ri` 所引用对象的值。注意，此时绑定的对象是一个临时量而非 `dval`。程序员既然让 `ri` 引用 `dval`，就肯定想通过 `ri` 改变 `dval` 的值，否则干什么要给 `ri` 赋值呢？如此看来，既然大家基本上不会想着把引用绑定到临时量上，C++ 语言也就把这种行为归为非法。

对 const 的引用可能引用一个并非 const 的对象

必须认识到，常量引用仅对引用可参与的操作做出了限定，对于引用的对象本身不是一个常量未作限定。因为对象也可能是个非常量，所以允许通过其他途径改变它的值：

```
int i = 42;
int &r1 = i;           // 引用 r1 绑定对象 i
const int &r2 = i;     // r2 也绑定对象 i，但是不允许通过 r2 修改 i 的值
r1 = 0;               // r1 并非常量，i 的值修改为 0
r2 = 0;               // 错误：r2 是一个常量引用
```

r2 绑定（非常量）整数 i 是合法的行为。然而，不允许通过 r2 修改 i 的值。尽管如此，i 的值仍然允许通过其他途径修改，既可以给 i 赋值，也可以通过像 r1 一样绑定到 i 的其他引用来修改。



2.4.2 指针和 const

与引用一样，也可以令指针指向常量或非常量。类似于常量引用（参见 2.4.1 节，第 54 页），指向常量的指针（pointer to const）不能用于改变其所指对象的值。要想存放常量对象的地址，只能使用指向常量的指针：

```
const double pi = 3.14; // pi 是个常量，它的值不能改变
double *ptr = &pi;      // 错误：ptr 是一个普通指针
const double *cptr = &pi; // 正确：cptr 可以指向一个双精度常量
*cptr = 42;             // 错误：不能给 *cptr 赋值
```

2.3.2 节（第 47 页）提到，指针的类型必须与其所指对象的类型一致，但是有两个例外。第一种例外情况是允许令一个指向常量的指针指向一个非常量对象：

```
double dval = 3.14; // dval 是一个双精度浮点数，它的值可以改变
cptr = &dval;       // 正确：但是不能通过 cptr 改变 dval 的值
```

63

和常量引用一样，指向常量的指针也没有规定其所指的对象必须是一个常量。所谓指向常量的指针仅仅要求不能通过该指针改变对象的值，而没有规定那个对象的值不能通过其他途径改变。



试试这样想吧：所谓指向常量的指针或引用，不过是指针或引用“自以为是”罢了，它们觉得自己指向了常量，所以自觉地不去改变所指对象的值。

const 指针

指针是对象而引用不是，因此就像其他对象类型一样，允许把指针本身定为常量。常量指针（const pointer）必须初始化，而且一旦初始化完成，则它的值（也就是存放在指针中的那个地址）就不能再改变了。把 * 放在 const 关键字之前用以说明指针是一个常量，这样的书写形式隐含着一种意味，即不变的是指针本身的值而非指向的那个值：

```
int errNum = 0;
int *const curErr = &errNum; // curErr 将一直指向 errNum
const double pi = 3.14159;
const double *const pip = &pi; // pip 是一个指向常量对象的常量指针
```

如同 2.3.3 节（第 52 页）所讲的，要想弄清楚这些声明的含义最行之有效的办法是从右向左阅读。此例中，离 curErr 最近的符号是 const，意味着 curErr 本身是一个常量对象，对象的类型由声明符的其余部分确定。声明符中的下一个符号是*，意思是 curErr

是一个常量指针。最后，该声明语句的基本数据类型部分确定了常量指针指向的是一个 int 对象。与之相似，我们也能推断出，pip 是一个常量指针，它指向的对象是一个双精度浮点型常量。

指针本身是一个常量并不意味着不能通过指针修改其所指对象的值，能否这样做完全依赖于所指对象的类型。例如，pip 是一个指向常量的常量指针，则不论是 pip 所指的对象值还是 pip 自己存储的那个地址都不能改变。相反的，curErr 指向的是一个一般的非常量整数，那么就完全可以用 curErr 去修改 errNumb 的值：

```
*pip = 2.72;           // 错误: pip 是一个指向常量的指针
                        // 如果 curErr 所指的对象 (也就是 errNumb) 的值不为 0

if (*curErr) {
    errorHandler();
    *curErr = 0;        // 正确: 把 curErr 所指的对象的价值重置
}
```

2.4.2 节练习

练习 2.27: 下面的哪些初始化是合法的？请说明原因。

- | | |
|-------------------------------|--------------------------------|
| (a) int i = -1, &r = 0; | (b) int *const p2 = &i2; |
| (c) const int i = -1, &r = 0; | (d) const int *const p3 = &i2; |
| (e) const int *p1 = &i2; | (f) const int &const r2; |
| (g) const int i2 = i, &r = i; | |

练习 2.28: 说明下面的这些定义是什么意思，挑出其中不合法的。

- | | |
|----------------------------|--------------------------|
| (a) int i, *const cp; | (b) int *p1, *const p2; |
| (c) const int ic, &r = ic; | (d) const int *const p3; |
| (e) const int *p; | |

练习 2.29: 假设已有上一个练习中定义的那些变量，则下面的哪些语句是合法的？请说明原因。

- | | |
|---------------|---------------|
| (a) i = ic; | (b) p1 = p3; |
| (c) p1 = ⁣ | (d) p3 = ⁣ |
| (e) p2 = p1; | (f) ic = *p3; |

2.4.3 顶层 const



如前所述，指针本身是一个对象，它又可以指向另外一个对象。因此，指针本身是不是常量以及指针所指的是不是一个常量就是两个相互独立的问题。用名词**顶层 const** (top-level const) 表示指针本身是个常量，而用名词**底层 const** (low-level const) 表示指针所指的对象是一个常量。

64

更一般的，顶层 const 可以表示任意的对象是常量，这一点对任何数据类型都适用，如算术类型、类、指针等。底层 const 则与指针和引用等复合类型的基本类型部分有关。比较特殊的是，指针类型既可以是顶层 const 也可以是底层 const，这一点和其他类型相比区别明显：

```
int i = 0;
int *const p1 = &i;      // 不能改变 p1 的值，这是一个顶层 const
const int ci = 42;       // 不能改变 ci 的值，这是一个顶层 const
const int *p2 = &ci;     // 允许改变 p2 的值，这是一个底层 const
```

```
const int *const p3 = p2; // 靠右的 const 是顶层 const, 靠左的是底层 const
const int &r = ci;        // 用于声明引用的 const 都是底层 const
```



当执行对象的拷贝操作时, 常量是顶层 const 还是底层 const 区别明显。其中, 顶层 const 不受什么影响:

```
i = ci;           // 正确: 拷贝 ci 的值, ci 是一个顶层 const, 对此操作无影响
p2 = p3;          // 正确: p2 和 p3 指向的对象类型相同, p3 顶层 const 的部分不影响
```

执行拷贝操作并不会改变被拷贝对象的值, 因此, 拷入和拷出的对象是否是常量都没什么影响。

另一方面, 底层 const 的限制却不能忽视。当执行对象的拷贝操作时, 拷入和拷出的对象必须具有相同的底层 const 资格, 或者两个对象的数据类型必须能够转换。一般来说, 非常量可以转换成常量, 反之则不行:

65

```
int *p = p3;           // 错误: p3 包含底层 const 的定义, 而 p 没有
p2 = p3;               // 正确: p2 和 p3 都是底层 const
p2 = &i;               // 正确: int* 能转换成 const int*
int &r = ci;            // 错误: 普通的 int& 不能绑定到 int 常量上
const int &r2 = i;       // 正确: const int& 可以绑定到一个普通 int 上
```

p3 既是顶层 const 也是底层 const, 拷贝 p3 时可以在乎它是一个顶层 const, 但是必须清楚它指向的对象得是一个常量。因此, 不能用 p3 去初始化 p, 因为 p 指向的是一个普通的 (非常量) 整数。另一方面, p3 的值可以赋给 p2, 是因为这两个指针都是底层 const, 尽管 p3 同时也是一个常量指针 (顶层 const), 仅就这次赋值而言不会有什么影响。

2.4.3 节练习

练习 2.30: 对于下面的这些语句, 请说明对象被声明成了顶层 const 还是底层 const?

```
const int v2 = 0;      int v1 = v2;
int *p1 = &v1, &r1 = v1;
const int *p2 = &v2, *const p3 = &i, &r2 = v2;
```

练习 2.31: 假设已有上一个练习中所做的那些声明, 则下面的哪些语句是合法的? 请说明顶层 const 和底层 const 在每个例子中有何体现。

```
r1 = v2;
p1 = p2; p2 = p1;
p1 = p3; p2 = p3;
```



2.4.4 constexpr 和常量表达式

常量表达式 (const expression) 是指值不会改变并且 在编译过程 就能得到计算结果的表达式。显然, 字面值属于常量表达式, 用常量表达式初始化的 const 对象也是常量表达式。后面将会提到, C++ 语言中有几种情况下是要用到常量表达式的。

一个对象 (或表达式) 是不是常量表达式由它的数据类型和初始值共同决定, 例如:

```
const int max_files = 20;           // max_files 是常量表达式
const int limit = max_files + 1;    // limit 是常量表达式
int staff_size = 27;               // staff_size 不是常量表达式
```




```
const int sz = get_size();           // sz 不是常量表达式
```

尽管 `staff_size` 的初始值是个字面值常量，但由于它的数据类型只是一个普通 `int` 而非 `const int`，所以它不属于常量表达式。另一方面，尽管 `sz` 本身是一个常量，但它的具体值直到运行时才能获取到，所以也不是常量表达式。

constexpr 变量

在一个复杂系统中，很难（几乎肯定不能）分辨一个初始值到底是不是常量表达式。当然可以定义一个 `const` 变量并把它的初始值设为我们认为的某个常量表达式，但在实际使用时，尽管要求如此却常常发现初始值并非常量表达式的情况。可以这么说，在此种情况下，对象的定义和使用根本就是两回事儿。

C++11 新标准规定，允许将变量声明为 **constexpr** 类型以便由编译器来验证变量的值是否是一个常量表达式。声明为 `constexpr` 的变量一定是一个常量，而且必须用常量表达式初始化：

```
constexpr int mf = 20;           // 20 是常量表达式
constexpr int limit = mf + 1;    // mf + 1 是常量表达式
constexpr int sz = size();       // 只有当 size 是一个 constexpr 函数时
                                // 才是一条正确的声明语句
```

尽管不能使用普通函数作为 `constexpr` 变量的初始值，但是正如 6.5.2 节（第 214 页）将要介绍的，新标准允许定义一种特殊的 `constexpr` 函数。这种函数应该足够简单使得编译时就可以计算其结果，这样就能用 `constexpr` 函数去初始化 `constexpr` 变量了。

Best
Practices

一般来说，如果你认定变量是一个常量表达式，那就把它声明成 `constexpr` 类型。

字面值类型

常量表达式的值需要在编译时就得到计算，因此对声明 `constexpr` 时用到的类型必须有所限制。因为这些类型一般比较简单，值也显而易见、容易得到，就把它称为“字面值类型”（`literal type`）。

到目前为止接触过的数据类型中，算术类型、引用和指针都属于字面值类型。自定义类 `Sales_item`、IO 库、`string` 类型则不属于字面值类型，也就不能被定义成 `constexpr`。其他一些字面值类型将在 7.5.6 节（第 267 页）和 19.3 节（第 736 页）介绍。

尽管指针和引用都能定义成 `constexpr`，但它们的初始值却受到严格限制。一个 `constexpr` 指针的初始值必须是 `nullptr` 或者 0，或者是存储于某个固定地址中的对象。

6.1.1 节（第 184 页）将要提到，函数体内定义的变量一般来说并非存放在固定地址中，因此 `constexpr` 指针不能指向这样的变量。相反的，定义于所有函数体之外的对象其地址固定不变，能用来初始化 `constexpr` 指针。同样是在 6.1.1 节（第 185 页）中还将提到，允许函数定义一类有效范围超出函数本身的变量，这类变量和定义在函数体之外的变量一样也有固定地址。因此，`constexpr` 引用能绑定到这样的变量上，`constexpr` 指针也能指向这样的变量。

指针和 constexpr

必须明确一点，在 `constexpr` 声明中如果定义了一个指针，限定符 `constexpr` 仅

66

C++
11

67

对指针有效，与指针所指的对象无关：

```
const int *p = nullptr;      // p 是一个指向整型常量的指针
constexpr int *q = nullptr; // q 是一个指向整数的常量指针
```

p 和 q 的类型相差甚远，p 是一个指向常量的指针，而 q 是一个常量指针，其中的关键在于 `constexpr` 把它所定义的对象置为了顶层 `const`（参见 2.4.3 节，第 57 页）。

与其他常量指针类似，`constexpr` 指针既可以指向常量也可以指向一个非常量：

```
constexpr int *np = nullptr; // np 是一个指向整数的常量指针，其值为空
int j = 0;
constexpr int i = 42;        // i 的类型是整型常量
// i 和 j 都必须定义在函数体之外
constexpr const int *p = &i; // p 是常量指针，指向整型常量 i
constexpr int *pl = &j;      // pl 是常量指针，指向整数 j
```

2.4.4 节练习

练习 2.32：下面的代码是否合法？如果非法，请设法将其修改正确。

```
int null = 0, *p = null;
```

2.5 处理类型

随着程序越来越复杂，程序中用到的类型也越来越复杂，这种复杂性体现在两个方面。一是一些类型难于“拼写”，它们的名字既难记又容易写错，还无法明确体现其真实目的和含义。二是有时候根本搞不清到底需要的类型是什么，程序员不得不回过头去从程序的上下文中寻求帮助。

2.5.1 类型别名

类型别名（type alias）是一个名字，它是某种类型的同义词。使用类型别名有很多好处，它让复杂的类型名字变得简单明了、易于理解和使用，还有助于程序员清楚地知道使用该类型的真实目的。

有两种方法可用于定义类型别名。传统的方法是使用关键字 `typedef`：

```
typedef double wages;      // wages 是 double 的同义词
typedef wages base, *p;    // base 是 double 的同义词，p 是 double* 的同义词
```

68 其中，关键字 `typedef` 作为声明语句中的基本数据类型（参见 2.3 节，第 45 页）的一部分出现。含有 `typedef` 的声明语句定义的不再是变量而是类型别名。和以前的声明语句一样，这里的声明符也可以包含类型修饰，从而也能由基本数据类型构造出复合类型来。

**C++
11**

新标准规定了一种新的方法，使用别名声明（alias declaration）来定义类型的别名：

```
using SI = Sales_item;    // SI 是 Sales_item 的同义词
```

这种方法用关键字 `using` 作为别名声明的开始，其后紧跟别名和等号，其作用是把等号左侧的名字规定成等号右侧类型的别名。

类型别名和类型的名字等价，只要是类型的名字能出现的地方，就能使用类型别名：

```
wages hourly, weekly;    // 等价于 double hourly, weekly;
```